
The Hangman-Game

Fredrik Strömbäck

2019-02-08

|Contents

1) Revision History	3
2) General Information	4
3) Vision	5
4) Project Plan	6
4.1 Introduction	6
4.2 Justification	6
4.3 Stakeholders	6
4.4 Resources	6
4.5 Hard- Software Requirements	7
4.6 Overall Project Schedule	7
4.7 Scope, Constraints and Assumptions	7
5) Iterations	8
5.1 Iteration 1	8
5.2 Iteration 2	9
5.3 Iteration 3	10
5.4 Iteration 4	11
6) Risk Analysis	12
6.1 List of risks	12
6.2 Strategies	12
7) Time Log	13
7.1 Time log iteration 1	13
7.2 Time log iteration 2	14
7.3 Time log iteration 3	15
7.4 Time log iteration 4	16
8) Reflection and The Project	17

1 | Revision History

Date	Version	Description	Author
2019-02-03	1.0	Starting up the planing of the project for exam one.	Fredrik Strömbäck
2019-02-11	1.1	Adds the planing for iteration two and creates a time log for it.	Fredrik Strömbäck
2019-03-08	1.2	Adds planing for the testing iteration	Fredrik Strömbäck
2019-03-11	1.3	Adds planing, design and test for the high-score feature	Fredrik Strömbäck

2 | General Information

Project Summary	
Project Name: The hangman-game	Project ID: fs222uv_1dv600_hangman
Project Manager: Fredrik Strömbäck	Main Client: Teachers of the course 1DV600
Key Stakeholders: Fredrik Strömbäck and the teachers of the course.	
Executive Summary: This is the project plan for the game that I am making for the course I am taking. This is made for the purpose of learning how to manage a bigger software project.	

3 | Vision

I will create a text-based hangman-game with JavaScript that the user/player could run in the nodejs-platform.

The game should start with a number of alternatives for the player to choose from. Here the player should be able to start the game, see the high-score list and exit the game, the choice is made by entering a number representing the different choices in the list presented.

When the user chooses to start the game, he/she will be asked to enter a player name, this will be saved in the high-score list if he/she can guess the word before the stick-figure is hanged.

After the user entered his/her name, a word from a list of words is chosen by random, this word is split up into individual letters but are displayed as underlines.

The user is now asked to enter a single letter, if the letter is present in the word chosen, every position with that letter should change from the underline to the letter entered. If the letter entered by the user is not in the word chosen, a part of the “hang-man” is displayed, for example the vertical pole. When the whole hangman is displayed, the user have lost the game and a new menu will appear and the user could start the game over.

What sets my game apart from other hangman games is that I would also want to add a local multiplayer mode, where another person could enter a word instead of that the word is chosen from the list of words. If the player completes the word they won’t appear in the high-score list because that would not be fair. This is so that two people can play against each other on the same machine.

Reflection

I feel that it’s very hard to think big with this project because the project itself is pretty small. Therefore the content of the vision document may be too specific. I can absolutely see the point of creating one though because this gives the person reading the document a good look of what the goals with the project are, without promising anything, and it is made clear for the client that may not understand the technology behind the project.

And the only one in the development team is me and I am already sure of what is being created, that makes this a little harder to write as well. I tried to let other people read it and all three of them (both other students and non students) understood what I wanted to create without me explaining it before they read this document.

4 | Project Plan

For the first iteration I will plan the project and write the vision document, try to plan for the following iterations, do the risk analysis and create a time log for each task in the iteration. I won't do any implementation of any code. I will however create some modules as skeleton-code, setup a package.json and install needed node modules, so when it's time to start implementing features I won't have to spend time doing that. I will also setup the application so it could be started with the "npm start"- command.

For now my plan is to create this game using or reusing modules/parts of a modules from other projects that I created. Each feature will get it's own module so the code in the modules could easily be maintained and changed.

Creating an object for each letter in the word chosen and push into an array to give the word the functionality needed, is my first thought for creating this game. The array of letters should then be displayed on the screen for the player.

How and if I will create the objects, if I use classes or other methods, and what functionality should be in what module will be decided in theme two where I will design the application using UML. So when it's time for iteration two my initial plan might change. So in every iteration I will have to update the project plan.

After that I will plan and do some testing, more details on this in iteration three.

4.1 Introduction

This is the assignment where I will create a text based hangman game for the course Software Tech (1DV600) I'm taking on Linnaeus University to become a web developer.

4.2 Justification

This application should be made for me to practice, so that I could use what I'm learning in this course when I'm being a part of or developing bigger projects/applications in the future. To be able to plan, design and test applications/systems is extremely important when developing bigger projects.

4.3 Stakeholders

Me and my teachers are the only stakeholder for this project, for me to pass this course I need this project to success, and of course my teachers have an interest of seeing that as many students as possible pass this course.

4.4 Resources

I a total of 9 weeks to complete this project. The course I am taking is at 50% speed so 20 hour a week.

Result: 180 hours to complete this project.

4.5 Hard- and Software Requirements

The application is developed on both Linux and Windows machines, the software is developed in Visual Studio Code and the application will run in nodejs.

I use the toggl web-app to track my time.

4.6 Overall Project Schedule

Deadlines:

Hand-in 1: 8 February

Hand-in 2: 21 February

Hand-in 3: 8 March

4.7 Scope, Constraints and Assumptions.

Scope:

In this project I will add a menu system for the hangman-game, a name entering feature, a feature to write and read a high-score list. And if there is time I would also like to add a simple multiplayer mode.

The application should also be able to print out a “hangman”-figure to show that the player are not guessing the correct letters.

The game should be able to start with the “npm-start”- command

There will not be any graphical interface because the game should be completely text based.

There will not be any part of the application where the player can see how a hangman game is played.

Constraints:

Time is a factor here, I only have a finite number of hours to do all this. I’m also new to developing for the nodejs-platform and that will absolutely set some limit of what I can do in the time available.

Assumptions:

I assume that the player knows how a simple game of hangman works.

Also that the user has some experience navigating in a text based environment (like a terminal) with only options presented to him/her.

Reflection

So far I think the project plan was the hardest part to write, because I am not completely sure of what to come in the following themes because this is the first time for me doing all of this. I can only plan for the first iteration right now and it’s pretty much just writing down everything I know so far about the project, analyze and reflect.

But as I said in the reflection of the vision document, I see the point of doing this too (especially when working with bigger projects and in bigger teams) but when I understand what designing with UML is and how testing actually works. But even then it would be hard to plan everything in detail before you are there, but at least then I could do better estimations of what would be included in each part, based on earlier experiences.

5 | Iterations

5.1 Iteration 1

Due date for iteration 1: 8th Feb 2019

The first iteration should be planed in detail.

I need to do everything in this iteration to pass this assignment.

Everything else is an addition.

The first iteration is the planing and setting up everything for the coming iterations. The iteration is split up into smaller tasks (see table down below), for easily managing.

Task	Responsible	Estimated Time
Set up git repo.	F.S	30 min
Create a skeleton document.	F.S	30 min
Write Vision Document.	F.S	1 hour
Write Project Plan.	F.S	1 hour
Plan iteration 2-4.	F.S	30 min
Risk Analysis.	F.S	2 hours
Writing skeleton-code and setup test modules.	F.S	1 hours
Fix start scripts in package.json	F.S	15 min

5.2 Iteration 2

Due date for iteration 1: 21th Feb 2019

Here I will design the classes for the application and learn how to write use cases.

Task	Responsible	Estimated Time
Plan iteration 2.	F.S	30 minutes
Watch Lectures.	F.S	10 hours
Read book and answer study material.	F.S	7 hours
Expand the use case diagram.	F.S	2 hours
Write fully dressed use case for “Play Game”.	F.S	3 hours
Make a state machine for “Play game”.	F.S	2 hours
Implement so the game is playable.	F.S	5 hours
Create class diagram.	F.S	3 hours

5.3 Iteration 3

Due date for iteration 1: 8th Mar 2019

This iteration is to learn how to plan, write and run both manual and automated tests for the application.

Task	Responsible	Estimated Time
Watch lectures	F.S	8 hours
Read chapter 8	F.S	1 hour 30 min
Writing test plan	F.S	2 hours
Writing manual test cases	F.S	2 hours
Running manual tests	F.S	15 min
Writing and running automated tests	F.S	4 hours
Write test report	F.S	2 hours

5.4 Iteration 4

Due date for iteration 1: 22th Mar 2019

Here I will add the feature for the gamer to check a high-score list that will be a local JSON file. I will also make tests to see that the feature is working properly. The documentation for the application will also be completed.

I've made a choice to only write manual tests for this last part because I feel they were more effective and gave me more info about how the game actually works than the automated tests I'm capable of writing.

All other files will be added to a separate folder to be appendices to this document.

See appendix folder for all appendices to this iteration.

Task	Responsible	Estimated Time
Write a use case for the "Check high-score" scenario.	F.S	2 hours
Make state machine for "Check high-score".	F.S	1 hour
Implement code for the feature.	F.S	3 hours
Expand the class diagram.	F.S	1 hour
Expand test plan.	F.S	1 hour
Writing manual tests for new feature.	F.S	2 hours
Running manual tests.	F.S	15 min
Writing test report	F.S	15 min
Complete the project plan.	F.S	2 hours
Finish the documentation structure	F.S	30 min

6 | Risk Analysis

6.1 List of risks

Risk	Probability	Impact
Computer/disk failure.	Low	Tolerable
The required time is underestimated.	High	Serious
Lack of skills to complete the project.	Moderate	Tolerable
Illness just before deadlines.	Moderate	Serious
The modules/code I plan to reuse doesn't work.	High	Insignificant

6.2 Strategies

Risk	Strategy
Computer/disk failure.	Always backup all the files both on Github and locally on other machines.
The required time is underestimated.	Plan each task with some slack.
Lack of skills to complete the project.	Ask for help from other students and/or teachers, when I get stuck and can't find solutions on my own.
Illness just before deadlines.	Try to complete each iteration in good time so if I get ill I still have time to finish the iteration in time for the deadline.
The modules/code I plan to use doesn't work.	"Repair" the modules or build new modules from scratch.

Reflection

This was absolutely the easiest part for me, I feel that I have more experience here, from earlier projects, what some risks are and I know how to minimize the impact if/when they occur. Here I feel that the size of the project limits the risks, for the good and the bad (good because not much could affect the project/product and I got no business to be affected. Bad because that I won't get as much practice to analyze and strategize). I think that I have to add more risks later when gain more understanding for the following themes, and the risk will be more specific to this project with that information.

7 | Time Log

7.1 Time log for iteration 1

Task	Responsible	Est. Time	Actual Time (15 min is minimum unit)
Set up git repo	F.S	30min	15 min*
Create skeleton document	F.S	30min	1 hour*
Write Vision document	F.S	1 hour	2 hours
Write Project Plan	F.S	1 hour 30 min	2 hours 30 min
Plan iteration 2-4	F.S	30 min	30 min
Risk Analysis	F.S	2 hours	1 hour 30 min
Writing skeleton-code and setup test modules.	F.S	1 hours	45 min
Fix start scripts in package.json	F.S	15 min	15 min

* = Estimation of the actual time. Forgot to time the task from the start.

And the time estimates are smaller than they should be because I realized too late that we should include the time for us learning/reading up on how to write a project plan.

The reason almost every task took longer that I estimated was because I clearly underestimated the time I needed to complete the task. I had to research while I was writing the tasks that I found hard to complete.

There are parts were the time was overestimated too, these were the tasks where I had experience from before, for example I've used Github in every course.

7.2 Time log for iteration 2

Task	Responsible	Est. Time	Actual Time (15 min is minimum unit)
Plan iteration 2.	F.S	30 min	45 min
Watch Lectures.	F.S	10 hours	13 hours
Read book and answer study material.	F.S	7 hours	9 hours*
Expand the use case diagram.	F.S	2 hours	1 hour 30 min
Write fully dressed use case for “Play Game”.	F.S	3 hours	2 hours 30 min
Make a state machine for “Play game”.	F.S	2 hours	2 hours 15 min
Implement so the game is playable.	F.S	5 hours	4 hours 15 min
Create class diagram.	F.S	3 hours	1 hour

The lecture task was underestimated by me and it was because I forgot to take the live lectures into account.

*Read the book task: I forgot to start the timer when I was reading in the book, so this time is an estimation. I know that it took more 7 hours, in total, because I had to read some parts more than once and some of the study questions I had to do some own research to find the answer to.

The class diagram I overestimated, I thought that the game would be bigger. I even included the time for me watching some videos when doing the diagram.

But I could be that I don't get how to do the diagram anyways so that the small diagram I made is wrong and that's why it didn't take more than an hour to make.

Reflection

This was the most fun part of all the themes so far, learning how to use a testing framework and writing my own tests was fun and gave me a good learning experience. The easiest part was to write all the manual test, because this is the way I've been testing my apps to this point. The hardest parts was to come up with all different tests when the application is so small, and I noticed that I hadn't really written my code so that it would be easy to do meaningful automated tests.

7.3 Time log for iteration 3

Task	Responsible	Estimated Time	Actual Time (15 min is minimum unit)
Watch lectures	F.S	8 hours	8 hours 45min
Read chapter 8	F.S	1 hour 30 min	1 hour
Writing test plan	F.S	2 hours	45 min
Writing manual test cases	F.S	2 hours	2 hours 45 min
Running manual tests	F.S	15 min	15 min
Writing and running automated tests	F.S	4 hours	5hours 15min
Write test report	F.S	2 hours	30min

Reflection

This was the most fun part of all the themes so far, learning how to use a testing framework and writing my own tests was fun and gave me a good learning experience. The easiest part was to write all the manual test, because this is the way I've been testing my apps to this point. The hardest parts was to come up with all different tests when the application is so small, and I noticed that I hadn't really written my code so that it would be easy to do meaningful automated tests.

7.4 Time log for iteration 4

Task	Responsible	Est. Time	Actual Time (15 min is minimum unit)
Write a use case for the “Check high-score” scenario.	F.S	2 hours	1 hour
Make state machine for “Check high-score”.	F.S	1 hour	30 min
Implement code for the feature.	F.S	3 hours	3 hours 30min
Expand the class diagram.	F.S	1 hour	30 min
Expand test plan.	F.S	1 hour	1 hour
Writing manual tests for new feature.	F.S	2 hours	45 min
Running manual tests.	F.S	15 min	15 min
Writing test report	F.S	15 min	15 min
Complete the project plan.	F.S	2 hours	1 hour 15 min
Finish the documentation structure	F.S	30 min	

8. Reflection Iteration 4 And The Project

Also see the appendixes for updated: Class diagram, state machines and test document.

This project/course was very educational. There were a lot of new things that I think different about today with how I write my code.

The hardest part was with all the documentation. How much to write, what to write and where to write it, estimate the time for each task in an iteration.

The code itself was solved in a couple of days.

When I wrote the vision in the first iteration I wanted to add a simple multiplayer feature, this was the realized quickly that I had to skip. There were not enough time to do this with my coding skill because I had to put the time I had left over on documentation. This was very unfortunate because I really wanted to add this feature, well now I have something to do in the summer.

In the fourth/last iteration I added the high-score feature at least. This was a challenge because this is the first time I used the “fs”-module this extensively.

And I decided that I was only going to use manual tests for this part, because, as I tested, in iteration 3 I think automated tests are unnecessary is an application this small(at least with my knowledge and skills). I feel that manual tests gave me more info about this small system.

And I wanted to write more testable code for the last module but, the rest of the application was not written this way so it was hard to make them work together in a good way.

The way I structured my files, with everything separated, I think is easier, therefore I put every file except the project plan in the appendix folder. I feel that this is easier to navigate in, instead of having everything in one single file.